# 项目完成报告

# 大作业一项目完成报告

# 项目仓库

GitHub 地址：https://github.com/shixuhui-ops/2026-ECNU-PJ1-1-questionnaire-system

---

## 一、项目目标

本项目开发一个基于 MongoDB 的在线问卷系统，实现用户注册登录、问卷创建管理、题目配置、条件跳转、问卷填写和结果统计。

## 二、系统设计

后端使用 Node.js 和 Express，数据库使用 MongoDB，认证使用 JWT，密码使用 bcryptjs 加密保存，前端使用 HTML、CSS 和 JavaScript。

系统主要模块：

1. 用户认证模块
2. 问卷管理模块
3. 题目和校验模块
4. 条件跳转模块
5. 问卷填写模块
6. 统计分析模块

# 三、MongoDB 设计

## users 集合

保存用户名、密码哈希、创建时间和更新时间。

## surveys 集合

保存问卷所有者、标题、说明、访问码、状态、截止时间、题目数组和跳转规则数组。

## responses 集合

保存问卷编号、填写用户、是否匿名、回答数组和提交时间。

选择 MongoDB 的原因：

- 问卷题目类型具有多样性
- 不同题目的校验字段不同
- 跳转规则适合嵌入问卷文档
- 文档结构可以减少复杂关联查询
- MongoDB 便于后续扩展题目类型

## 字段与索引说明

### users 集合

| 字段 | 类型 | 说明 |
| --- | --- | --- |
| `_id` | ObjectId | 用户主键 |
| `username` | String | 用户名，唯一索引 |
| `passwordHash` | String | bcrypt 密码哈希 |
| `createdAt` | Date | 注册时间 |
| `updatedAt` | Date | 修改时间 |

### surveys 集合

| 字段 | 类型 | 说明 |
| --- | --- | --- |
| `_id` | ObjectId | 问卷主键 |
| `ownerId` | ObjectId | 创建者编号，建立索引 |
| `title` | String | 问卷标题 |

| 字段 | 类型 | 说明 |
| --- | --- | --- |
| accessCode | String | 公开访问码，唯一索引 |
| status | String | draft、published 或 closed |
| deadline | Date | 截止时间 |
| questions | Array | 嵌入式题目文档 |
| jumpRules | Array | 嵌入式跳转规则文档 |

**responses 集合**

| 字段 | 类型 | 说明 |
| --- | --- | --- |
| _id | ObjectId | 提交记录主键 |
| surveyId | ObjectId | 所属问卷，建立索引 |
| respondentId | ObjectId/null | 登录用户编号 |
| anonymous | Boolean | 是否匿名提交 |
| answers | Array | 题目答案数组 |
| submittedAt | Date | 提交时间，建立索引 |

## 嵌入与引用选择

题目和跳转规则嵌入 surveys 文档，因为它们只属于某一份问卷，读取问卷时通常需要同时读取这些内容，嵌入可以减少关联查询。

问卷所有者、提交用户和提交记录使用引用关系，因为用户和回答记录可能独立增长，并且统计时需要按问卷筛选大量提交数据。这样可以避免问卷文档无限增长，也便于对回答记录建立索引。

## MongoDB 索引

- users.username：保证用户名唯一并加快登录查询
- surveys.ownerId：加快查询当前用户创建的问卷
- surveys.accessCode：加快公开访问问卷
- surveys.status：加快筛选已发布问卷
- responses.surveyId：加快问卷统计查询
- responses.respondentId：支持按填写用户查询记录
- responses.submittedAt：支持按提交时间排序

# MongoDB 文档示例

## users 文档

```
{
  "_id": "ObjectId",
  "username": "testuser",
  "passwordHash": "$2b$12$...",
  "createdAt": "2026-10-15T12:00:00.000Z",
  "updatedAt": "2026-10-15T12:00:00.000Z"
}
```

## surveys 文档

```
{
  "_id": "ObjectId",
  "ownerId": "ObjectId",
  "title": "学生信息调查问卷",
  "description": "用于测试问卷功能",
  "accessCode": "2b5ddbf18f84",
  "allowAnonymous": true,
  "status": "published",
  "questions": [
    {
      "_id": "ObjectId",
      "type": "single",
      "title": "你的年级",
      "required": true,
      "options": [
        { "label": "大一", "value": "大一" },
        { "label": "大二", "value": "大二" }
      ]
    }
  ],
  "jumpRules": [
    {
      "sourceQuestionId": "ObjectId",
      "operator": "equals",
      "expectedValue": "大一",
      "targetQuestionId": "ObjectId"
    }
  ]
}
```

## responses 文档

```
{
  "_id": "ObjectId",
  "surveyId": "ObjectId",
  "respondentId": null,
  "anonymous": true,
  "answers": [
    {
      "questionId": "ObjectId",
      "value": "大一"
```

```
    },
    {
      "questionId": "ObjectId",
      "value": 20
    }
  ],
  "submittedAt": "2026-10-15T12:30:00.000Z"
}
```

### 系统业务流程

1. 用户注册或登录，后端返回 JWT。
2. 用户创建草稿问卷，系统生成唯一访问码。
3. 创建者添加题目、校验规则和条件跳转规则。
4. 问卷发布后，填写者通过访问码获取公开问卷。
5. 填写者按跳转规则逐题回答，系统实时进行答案校验。
6. 提交成功后，回答保存到 `responses` 集合。
7. 创建者通过统计接口查看回答数量、选项分布、文本内容和数字平均值。

## 四、关键业务逻辑

### 用户认证

注册时使用 bcryptjs 对密码进行哈希，登录成功后签发 JWT。需要权限的接口通过 Bearer Token 验证用户身份。

### 问卷管理

用户可以创建草稿问卷，添加题目和跳转规则。问卷至少包含一道题目后才能发布。问卷发布后可以通过访问码公开访问，关闭后不能继续填写。

### 题目类型

系统支持：

- 单选题
- 多选题
- 文本填空题
- 数字填空题

### 校验逻辑

系统支持必答校验、选项合法性校验、多选数量校验、文本长度校验、数字范围校验和整数校验。

### 跳转逻辑

跳转规则由问卷创建者动态配置。系统根据当前题目答案和比较运算符查找匹配规则，并返回下一道题目。

### 统计逻辑

统计接口根据已提交的回答动态生成：

- 单选题各选项数量
- 多选题各选项选择次数
- 文本题全部文本内容
- 数字题平均值和全部数字

## 五、测试结果

执行命令：

```
Set-Location .\backend
npm test
```

测试结果：

```
所有自动化测试通过
```

测试覆盖：

- 用户注册
- 创建问卷
- 添加题目
- 添加跳转规则
- 发布问卷
- 获取公开问卷
- 条件跳转
- 提交问卷
- 查看统计

### 失败场景测试

- 重复用户名注册：正确返回失败
- 无效单选答案：正确拒绝提交
- 未填写必答题：正确拒绝提交
- 非法数字答案：正确拒绝提交
- 多选数量超限：正确拒绝提交
- 未授权统计请求：正确拒绝访问

- 已关闭问卷提交：正确拒绝提交
- 超过截止时间提交：正确拒绝提交

### 实际功能演示结果

- 成功创建“学生信息调查问卷”
- 成功添加单选题“你的年级”
- 成功添加数字题“你的年龄”
- 成功配置“大一跳转到年龄题”
- 成功发布问卷并生成访问地址
- 成功按跳转规则完成填写
- 成功提交问卷
- 统计结果显示总回答数为 1
- 年级“大一”选择数为 1
- 年龄平均值为 20

完整测试用例、输入、预期输出和实际结果见 `TEST_CASES.md`。

## 六、遇到的问题及解决方法

### 1. npm 项目名称问题

项目根目录包含中文名称，直接执行 `npm init -y` 时失败。解决方法是在 `backend` 子目录中初始化 Node.js 项目。

### 2. MongoDB 服务未注册为 Windows 服务

系统中没有 MongoDB Windows 服务。解决方法是使用本地数据库目录手动启动：

```
mongod --dbpath .\backend\data\db
```

### 3. 自动化测试请求缺少 JSON 请求头

测试工具合并请求配置时覆盖了 `Content-Type`。调整请求配置合并顺序后解决。

### 4. nodemon 频繁重启

开发期间使用 `node src/server.js` 直接启动服务，避免 nodemon 监控文件造成重复重启。

## 七、AI 使用说明

AI 主要帮助完成：

- 数据库集合和字段设计
- Express 路由实现
- JWT 认证中间件
- 问卷校验和跳转逻辑
- 统计接口
- 自动化测试代码
- 项目说明文档

人工完成和修改：

- 根据课程要求确定功能范围
- 检查并调整 MongoDB 数据结构
- 增加权限和匿名填写逻辑
- 修复测试请求头问题
- 验证全部 API 流程
- 编写启动说明和测试说明

## 八、项目运行方式

1. 启动 MongoDB。
2. 在 `backend` 目录执行 `node src/server.js`。
3. 在 `frontend` 目录执行 `python -m http.server 5500`。
4. 浏览器访问 `http://localhost:5500`。